

1 Introduction

This report presents **Milestone 2**, focusing on **GraphSLAM**: simultaneously estimating the robot’s trajectory and constructing an environment map under real sensor and odometry noise. Building on Milestone 1’s optical flow pipeline, the robot traverses the environment, accumulates a constraint graph, and solves for the globally consistent set of poses and landmarks. Code: github.com/pepepepepordu/perception-of-cognitive-robot-project

2 Robot & Environment

Implemented in Webots R2025a in a rectangular arena with a static white box. The e-puck uses wheel encoders for odometry, PS0–PS7 proximity sensors, and an RPLidar A2 on the turret slot, driven via WASD keyboard control.

Grid cell size: Minimum cell size $n = \max(r, s)$ where r = robot movement resolution (≈ 1 mm) and s = sensor resolution (≈ 2 cm at close range for the RPLidar A2). Thus $n = 2$ cm; our system uses equivalent continuous-coordinate precision.

3 GraphSLAM: Building the Graph

3.1 Odometry Constraint

Encoder readings are converted to $[dx, dy]$ via differential drive kinematics. A pose node x_t is added every 5 cm of travel, linked by:

$$p(x_t | x_{t-1}, u_t) \propto \exp\left(-\frac{1}{2}(x_t - g(x_{t-1}, u_t))^T R_t^{-1}(\dots)\right)$$

with $\sigma_R = 0.05$ m.

3.2 Sensor Constraint & Landmark Detection

The lidar point cloud is filtered to $[0.1, 2.5]$ m and clustered (radius 0.2 m, min. 3 pts). Each centroid $[r_x, r_y]$ is transformed to world frame:

$$w_x = x + r_x \cos \theta - r_y \sin \theta, \quad w_y = y + r_x \sin \theta + r_y \cos \theta$$

with sensor noise $\sigma_Q = 0.1$ m.

3.3 Landmark Correspondence Function

A **LandmarkRegistry** implements nearest-neighbour matching in world frame. Correspondence is declared if $d < 0.3$ m ($= 3\sigma_Q$, a 3σ bound on sensor noise), merging observations of the same physical feature regardless of robot heading. Otherwise a new landmark is registered.

3.4 Graph Pruning

Two strategies are applied: (1) **Landmark-free pose removal** — poses with no landmark observation in the 5 cm window are not added, keeping the graph compact. (2) **Redundant landmark merging** — the registry updates landmark positions via running average, consolidating repeated detections of the same feature.

4 GraphSLAM: Solving the Graph

4.1 Linear Approximation

Taylor series linearization of the global error

$$E(\mu) = \sum_t e_{x_t}^T R_t^{-1} e_{x_t} + \sum_t \sum_i e_{z_t^i}^T Q_t^{-1} e_{z_t^i}$$

yields the information system $\Omega\mu = \xi$. Per constraint ($i \rightarrow j$), $w = 1/\sigma$:

$$\Omega_{ii} += w, \quad \Omega_{jj} += w, \quad \Omega_{ij} -= w, \quad \Omega_{ji} -= w$$

$$\xi_i += z \cdot w, \quad \xi_j -= z \cdot w$$

The matrix is pre-allocated at full size with landmark indices fixed after all pose slots. Pose 0 is anchored at origin with confidence 10^6 .

4.2 State Estimation & Loop Closure

$\mu^* = \arg \min E(\mu)$ is recovered via `numpy.linalg.solve` ($\Omega\mu = \xi$), with least-squares fallback. When the robot returns within 0.15 m of a past pose (min. gap 20 poses), a tight constraint ($\sigma = 0.01$ m) is inserted, correcting accumulated drift for global consistency.

5 Results

The e-puck was driven around the white box for one complete circuit. Multiple unique landmarks were registered across the arena walls and box surfaces, with the number varying per run depending on traversal path and lidar visibility. Loop closure events were consistently triggered upon returning near the start, visibly snapping the SLAM path into a globally consistent shape. Dead reckoning diverged noticeably over the full circuit, confirming the corrective value of the constraint graph. The live map updated every 5 cm, correctly distinguishing the robot’s traversed path from static landmark positions.

6 Future Work

Milestone 3 (3 April) focuses on **anchoring and planning**. The robot must predict the future positions of dynamic objects detected by the Milestone 1 optical flow pipeline and plan a collision-free path to a goal using the SLAM-constructed map. This requires classifying lidar detections as permanent obstacles (walls, box) vs. transient moving entities, predicting dynamic object trajectories, and integrating a path planner that avoids both static and predicted dynamic obstacles.